



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe Problema 1

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

24 febrero 2026

1. Método Utilizado

Para resolver numéricamente la ecuación $F(d) = 0$, se empleó el método de Brent, disponible en Python a través de la función `brentq` del módulo `scipy.optimize`. Este método combina internamente los algoritmos de bisección, secante e interpolación cuadrática inversa, lo que le otorga las siguientes características:

- Garantiza convergencia siempre que F cambie de signo en el intervalo $[a, b]$ proporcionado.
- No requiere el cálculo de la derivada de $F(d)$.
- Alcanza una precisión cercana a la tolerancia de máquina ($\approx 10^{-15}$).

Antes de aplicar el método, se realizó un análisis gráfico de $F(d)$ en el intervalo $(0, 19)$, lo cual permitió identificar visualmente la región donde la función cruza el eje d y confirmar que existe un cambio de signo en el intervalo $[1, 10]$. Dicho intervalo fue el utilizado como entrada para `brentq`.

Es importante destacar que el dominio válido de $F(d)$ está restringido a $0 < d < 2r = 20$, ya que el argumento de $\arccos(d/2r)$ debe pertenecer a $[-1, 1]$, y $g(d) \rightarrow 0$ cuando $d \rightarrow 20$, lo que provoca una división por cero en σ_c^2 . Por esta razón el intervalo de búsqueda se mantuvo alejado del límite superior.

2. Bibliotecas Empleadas

- **NumPy** (`import numpy as np`): biblioteca fundamental para cómputo numérico en Python. Se utilizó para funciones matemáticas como `np.log()`, `np.log10()` y `np.linspace()`, así como para la construcción de vectores mediante `np.array()`.
- **Math** (`import math`): módulo estándar de Python para operaciones matemáticas escalares. Se empleó `math.acos()`, `math.sqrt()` y `math.pi`, ya que estas funciones operan sobre valores individuales dentro de `func_d`.
- **Matplotlib** (`import matplotlib.pyplot as plt`): biblioteca de visualización. Se utilizó para graficar $F(d)$ en el intervalo de interés y trazar la línea de referencia $y = 0$ mediante `plt.axhline()`.
- **SciPy** (`from scipy.optimize import brentq`): biblioteca de métodos científicos y numéricos. Se empleó específicamente la función `brentq`, que implementa el método de Brent para la búsqueda de raíces en un intervalo dado.

3. Código Desarrollado y Solución Obtenida

Listing 1: Implementación en Python para la estimación de distancia entre sensores

```
1 import numpy as np
2 import math
3 import matplotlib.pyplot as plt
4 from scipy.optimize import brentq
5
6 def func_d(d):
7     alpha = 4
8     sigmadb = 4
9     tlambda = 1
10    r = 10
11    x1 = 7
12    x2 = 6
13
14    S = math.pi * r**2
15    k = (10 * alpha) / np.log(10)
16    gd = (2*S/math.pi) * math.acos(d/(2*r)) \
17        - d * math.sqrt(r**2 - d**2/4)
18    sigmaR2 = sigmadb**2 / (10*alpha)**2
19    sigmaC2 = (gd**2 / (2*tlambda*k**2)) * (1/gd + 1/S)
20
21    return np.log10(x1/d) / (sigmaR2*np.log(10)) + d*(x2-d)/sigmaC2
22
23 # --- Vectores para la grafica ---
24 d_vals = np.linspace(0.1, 19, 500)
25 y_vals = np.array([func_d(d) for d in d_vals])
26
27 # --- Grafica ---
28 plt.figure()
29 plt.plot(d_vals, y_vals, 'b-', linewidth=1.5)
30 plt.axhline(0, color='r', linestyle='--', linewidth=1.2)
31 plt.grid(True)
32 plt.xlabel('d')
33 plt.ylabel('F(d)')
34 plt.title('Funcion F(d)')
35 plt.show()
36
37 # --- Encontrar la raiz ---
38 d_star = brentq(func_d, 1, 10)
39 print(f'Distancia estimada: d* = {d_star:.10f}')
40
41 # --- Residuo |F(d*)| ---
42 residuo = abs(func_d(d_star))
43 print(f'Residuo |F(d*)| = {residuo:.2e}')
```

Al ejecutar el código anterior se obtuvieron los siguientes resultados:

Cantidad	Valor
Distancia estimada d^*	6,1919497069
Residuo $ F(d^*) $	$\approx 0,00 \times 10^0$

Tabla 1: Resultados obtenidos con el método de Brent.

La distancia estimada entre los sensores A y B es $d^* \approx 6,1919$ unidades, valor que coincide con el obtenido en la implementación de GNU Octave, lo que valida la correctitud de ambas soluciones. El residuo obtenido es prácticamente cero, lo que confirma que **brentq** encontró la raíz con una precisión cercana a la tolerancia de máquina.